

Distributed Caching

* Cache data is shared across multiple servers in a distributed system. Instead of

Server A $\rightarrow$ Local cache, Server B $\rightarrow$ Local cache

we use:

Server A & B $\rightarrow$ shared cache clusters.

All application instances use same cache layer.

Distributed caching Provides:

- $\rightarrow$ Horizontal scalability
- $\rightarrow$ Shared cache state
- $\rightarrow$ High availability
- $\rightarrow$ Better memory utilization
- $\rightarrow$ Fault tolerance.

Working

* Usually implemented using:

- Redis clusters
- Memcached cluster.

* Data is distributed across nodes using:

- consistent hashing.

* If one node fails, others continue serving.

Example

* E-commerce system:

- 10 backend servers
- 1 redis cluster

 } All servers use same Redis

- User session stored once.

- All backend instances can access it.

without distributed caching:

$\rightarrow$ User loggen in on one server might appear logged out on another.

Cache Replacement Policies

Assume :- cache memory is full, when new data comes what should we remove?

This decision is called cache replacement policies.

1) LRU (Least Recently Used) :- Remove the item that hasn't been used recently.

Assumption : Recently used data likely used again.

Most common policy.

2) LFU (Least Frequently Used) :- Remove item used least no. of times.
Good when access patterns are stable.

3) FIFO (First in first out)
Remove oldest inserted item. Simple but less intelligent.

4) TTL Based expiry :- Remove the item after fixed time.
useful for time-sensitive data.

Why these policies matter?

wrong policy can : 1) remove hot data
3) Increase DB load.

2) Reduce cache hit rate

Backend perspective

Large-scale system usually uses :

- Distributed Redis cluster
- Cache aside strategy
- LRU or LFU eviction.

This combo works well for most applications.

Cache Thrashing

- Cache works best when :
- frequently used data stays in memory.
 - Rarely used data is removed.

But sometimes systems behaves like :

- New data constantly replaces old data
- Cache keeps evicting useful items
- Hit rate becomes low

This situation is called 'cache Thrashing'.

* It happens when working dataset is larger than cache capacity, causing continuous evictions & low cache effectiveness.

Simple :- cache is too small for the workload.

So : Data enters cache → Immediately gets evicted → Re-loads again
↓
Evicted again

This causes :-

- High latency
- CPU spikes
- High DB load
- Poor system performance.

Example :- Cache size = 100 items, system frequently accesses 1000 of unique items randomly.

Result :- Almost every request → cache miss

This is 'thrashing'.

How to Prevent Cache Thrashing

- Increase cache size
- Improve replacement policies
- Cache only hot data
- Use better Key design
- Use separate Cache layers for different workloads.

Monitoring cache hit rate is critical.

AI Response Caching

Problem:- LLM calls are expensive, slow. If multiple user asks same question. Calling LLM every time is wasteful. So we can cache AI responses.

* AI response caching means :- storing previously generated LLM responses and returning them for similar or identical queries.

Challenges:- AI query queries are rarely identical.

User may ask:-
• Capital of France?
• What is France's capital?

Exact string matching fails

Advanced Approach

Use semantic similarity:

- Convert query to embedding
- Check if similar query exists
- If similarity above threshold $\rightarrow$ reuse answer.

Embedding Caching (study)

* Storing previously generated embeddings so they can be reused instead of recomputed.

- Types:-
- Query embedding cache
 - Document embedding cache
 - Version based embedding (combine with metadata filtering)
 - TTL based (Regenerate periodically)

AI caching Architecture

Layer 1 $\rightarrow$ Response cache

Layer 2 $\rightarrow$ Embedding cache

Layer 3 $\rightarrow$ Vector DB retrieval

Layer 4 $\rightarrow$ LLM

} Each layer reduces cost & latency.